

呈现财务大数据——财务数据可视化

目录

1 认识Matplotlib-数据可视化工具包

2 财务数据可视化-Pyecharts

01

认识Matplotlib-数据可视化工具包

Matplotlib简介

Matplotlib简介

Matplotlib是Python的第三方绘图库，仅需几行代码，就能够生成各种格式的图形(如：折线图、散点图、柱状图等等)。使用Matplotlib生成的图形质量较高，甚至可以达到出版级别。

执行步骤

第一步：打开开始菜单中“命令提示符”（快捷键win+R）

第二步：在命令提示符中输入如下代码



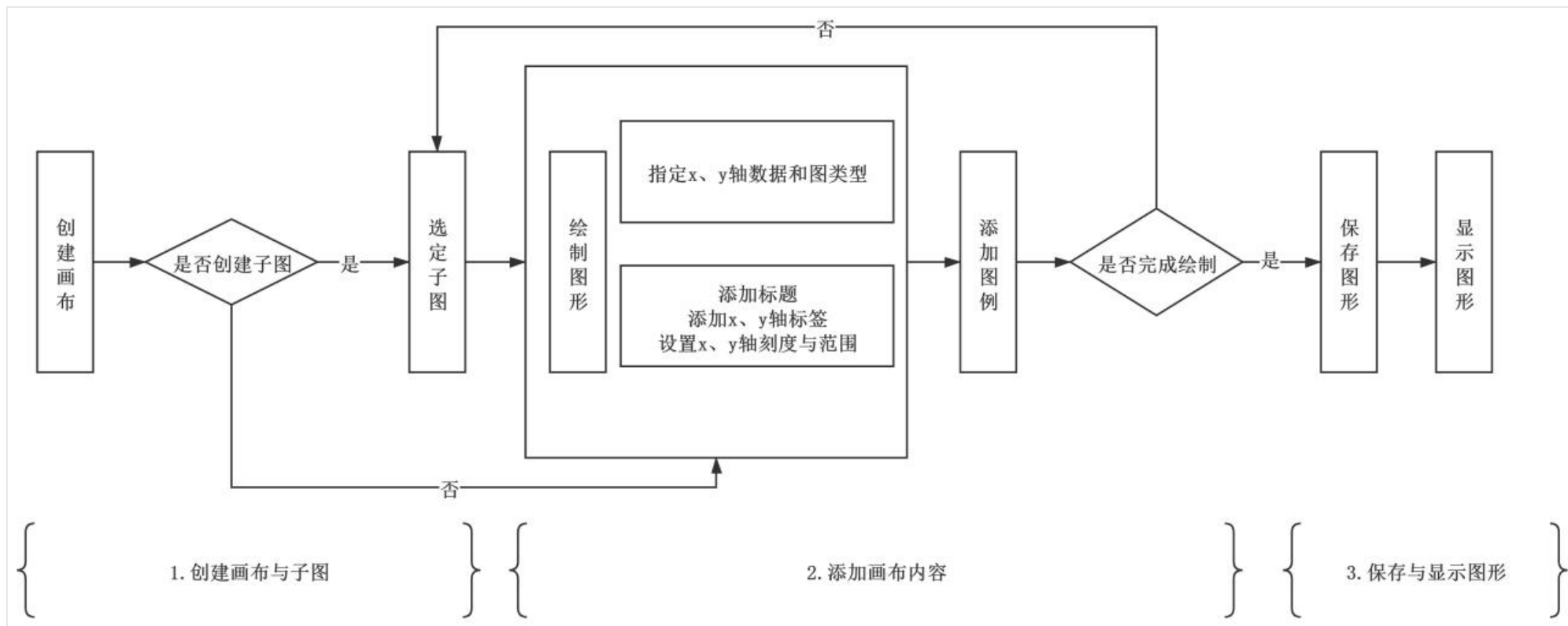
```
命令提示符
Microsoft Windows [版本 10.0.19042.1165]
(c) Microsoft Corporation。保留所有权利。

C:\Users\wu>pip3 install matplotlib
```

引入规则

Matplotlib最核心的模块是pyplot模块，几乎所有的2D图形都是通过该模块绘制，pyplot模块约定别名为plt。pyplot模块引入规则：`from matplotlib import pyplot as plt`

Matplotlib初级应用



设置字体



matplotlib默认情况不支持中文，可以使用以下简单的方法来解决。
函数语法如下：

```
plt.rcParams['font.family']='Microsoft YaHei'
```

因为matplotlib默认字体中是没有中文的，所以当我们使用中文做图形标签时，将会无法显示，会显示□□。解决这个问题需要在代码中加入如上语句。



创建画布

figure()函数：构建一张新的空白画布(返回一个figure对象)，函数语法如下：

```
plt.figure(num=None,figsize=None,dpi=None,facecolor=None,edgecolor=None,frameon=True,FigureClass=Figure,clear=False,**kwargs)
```

常用参数	说明
num	图像编号或名称，数字为编号，字符串为名称，是图形的唯一标识符
<u>figsize</u>	指定 figure 的宽和高，单位为英寸
dpi	指定绘图对象的分辨率
<u>facecolor</u>	背景颜色，默认白色

figure()函数常用参数

创建画布

例4-1-1：创建一个编号为“1”、宽度为9、高度均为5、分辨率为100的画布。

```
from matplotlib import pyplot as plt  
plt.figure(num="1",figsize=(9,5),dpi=100)
```

运行结果如下：

<Figure size 900x500 with 0 Axes>

<Figure size 900x500 with 0 Axes>

分辨率，又称解析度、解像度，可以细分为显示分辨率、图像分辨率、打印分辨率和扫描分辨率等。通常情况下，图像的分辨率越高，所包含的像素就越多，图像就越清晰，印刷的质量也就越好。同时，它也会增加文件占用的存储空间。Figure size 900x500则表明系统后台已建立一张分辨率为900×500的画布

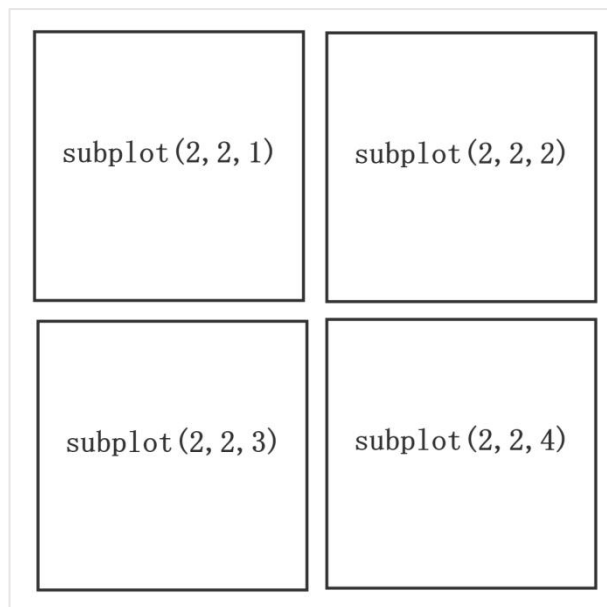
创建子图

subplot()函数：创建子图(即创建一个n行m列的axes对象)，并选择绘图区域。

语法：plt.subplot(nrows,ncols,index)

nrows、ncols：子图的行数和列数，表示绘图区域被分为n行m列；

index：子图索引，表示当前绘图区，子图按照从左到右、从上往下的顺序进行索引编号，编号从1开始

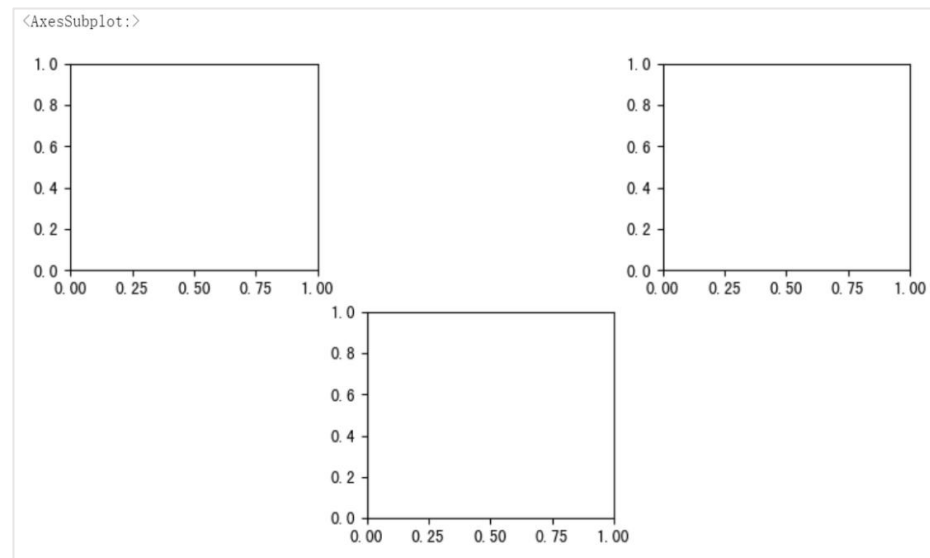


subplot()函数创建子图坐标系

创建子图

例4-1-2：在例4-1-1创建的画布上创建2行3列的子图。

```
from matplotlib import pyplot as plt
plt.figure(num="1",figsize=(9,5),dpi=100)
plt.subplot(2,3,1)
plt.subplot(2,3,3)
plt.subplot(2,3,5)
```



运行结果

注意：使用subplot可以规划figure划分n个子图，但每条subplot命令只会创建一个子图。

添加画布内容

添加画布内容是绘图的主体部分，首先，要确定x、y轴的数据及绘图类型，在Matplotlib中，大部分图形样式的绘制方法都存在于pyplot模块中，较常用图形如表所示：

图形	说明	图形	说明
<code>plt.plot()</code>	折线图	<code>plt.pie()</code>	饼状图
<code>plt.scatter()</code>	散点图	<code>plt.area()</code>	面积图
<code>plt.bar()</code>	柱状图	<code>plt.stackplot()</code>	<u>堆叠图</u>
<code>plt.hist()</code>	直方图	<code>plt.boxplot()</code>	箱线图

pyplot模块常用图形函数

添加画布内容

确定x、y轴的数据及绘图类型后，即可设置图形的标题，x、y轴的标签、刻度、范围等，最后添加图例。各类标签及图例常用函数如表所示：

函数	说明	函数	说明
<code>plt.title()</code>	设置图像标题	<code>plt.xlim()</code>	设置 x <u>轴范围</u>
<code>plt.xlabel()</code>	设置 x 轴名称	<code>plt.ylim()</code>	设置 y <u>轴范围</u>
<code>plt.ylabel()</code>	设置 y 轴名称	<code>plt.xticks()</code>	设置 x 轴刻度
<code>plt.legend()</code>	设置图例	<code>plt.yticks()</code>	设置 y 轴刻度

pyplot模块常用绘制函数

保存与显示图形

函数	说明
<code>plt.savefig()</code>	保存绘制的图形
<code>plt.show()</code>	显示图形

保存与显示函数

语法

```
import matplotlib.pyplot as plt  
""" 一些画图代码 """  
  
plt.savefig("filename.png")  
plt.show()
```

如果需要保存图片，必须在`plt.show()`之前调用`plt.savefig()`，这是由于调用`plt.show()`后会创建一个新的空白图片，在其后调用`plt.savefig()`将会保存新的空白图片。

注意

初级绘图

例4-1-3：甲公司2020年1-5月份销售额依次为45000元、38000元、54000元、55000元、48000元，使用pyplot模块绘制1-5月销售额柱状图

```
from matplotlib import pyplot as plt
# 步骤一：创建画布
plt.figure(figsize=(5,3),dpi=120)

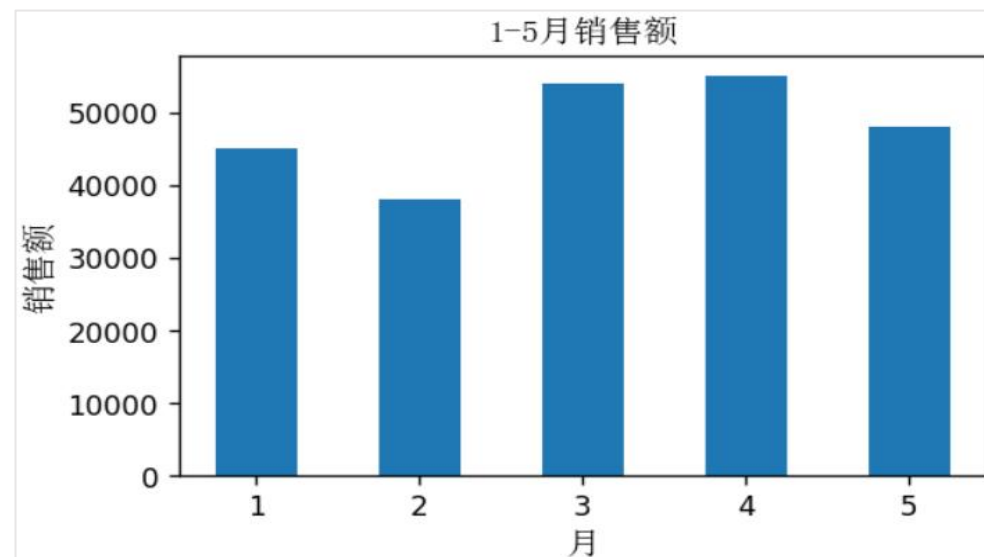
# 步骤二：创建绘图区(只有一个绘图区时可省略)
plt.subplot(1,1,1)

# 步骤三：指定数据和图形(x轴为月份，y轴为销售额)
x = [1,2,3,4,5]
y = [45000,38000,54000,55000,48000]
plt.bar(x,y,width=0.5)

# 步骤四：添加标签
plt.xlabel('月')
plt.ylabel('销售额')
plt.title('1-5月销售额')

# 步骤五：保存图形
plt.savefig('1-5月销售额.png')

# 步骤六：显示图形
plt.show()
```



运行结果

matplotlib可视化绘图-绘制折线图

折线图是最基本的图形，由线条组成。plot()函数用于绘制折线图。

语法：plt.plot(x, y, scalex=True, scaley=True, data=None, **kwargs)

常用参数	说明
<u>x, y</u>	表示 x、y 轴数据，接收数组、列表、元组等
<u>scalex, scaley</u>	是否自动缩放 x、y 轴，默认为 True
data	可索引对象(如: <u>dict</u> 、 <u>DataFrame</u>)，如果给定 data，则只需提供在 x、y 中绘制的标签名称，如以 <u>DataFrame</u> 中的列作为 x、y <u>轴数据</u>
color(c)	设置折线颜色，接收字符串
marker	设置线条上标记点的样式，默认 None，接收字符串
<u>linestyle(ls)</u>	设置线型的样式，默认实线'-'，接收字符串
<u>linewidth(lw)</u>	设置线型的宽度，接收数值
alpha	设置线型的透明度，0.0-1.0 之间
label	图例内容，接收字符串

折线图plot()函数常用参数

绘制折线图

color 参数设置	颜色	对应十六进制颜色码
color=' b'	蓝(blue)	color='#0000FF'
color=' g'	绿(green)	color='#008000'
color=' r'	红(red)	color='#FF0000'
color=' w'	白(white)	color='#FFFFFF'
color=' m'	洋红(magenta)	color='#FF00FF'
color=' y'	黄(yellow)	color='#FFFF00'
color=' k'	黑(black)	color='#000000'
color=' c'	青(cyan)	color='#00FFFF'

常用颜色参数

linestyle 参数设置	线型
linestyle=' -'	默认实线
linestyle=' --'	虚线
linestyle=' -. '	点划线
linestyle=' :'	点状线

常用线型参数

marker 参数设置	标记点	marker 参数设置	标记点
marker=' . '	实心点	marker=' +'	加号
marker=' s'	正方形	marker=' v'	一角朝下三角形
marker=' o'	圆圈	marker=' ^'	一角朝上三角形
marker=' *'	星号	marker=' D'	菱形
marker=' p'	五边形	marker=' H'	六边形

常用标记点参数

绘制折线图

例4-1-4: 读取data.xlsx文件, 根据“资产负债表项目”和率、总资产净利率, 并计算各期毛利率、营业净利率、权益净利率
URL地址: <https://keyun-oss.acctedu.com/app/bigdata>

注意: URL地址可以单独定义为变量, 方便切换不同数据源。Pandas中Merge函数可以简单的将同索引数据合并在一起。但如果数据索引不一致, 则需要用到DataFrame中的set_index函数重新设置索引。

```
# 引入pandas
import pandas as pd

# 读取资产负债表项目
df1 = pd.read_excel(r'https://keyun-oss.acctedu.com/app/bigdata/basics/data.xlsx',sheet_name = 0)

# 读取利润表项目
df2 = pd.read_excel(r'https://keyun-oss.acctedu.com/app/bigdata/basics/data.xlsx',sheet_name = 1)

# 调用merge函数连接df1、df2
df3 = pd.merge(df1,df2)

# 指标计算
df3['毛利率'] = (df3['营业收入']-df3['营业成本'])/df3['营业收入']
df3['营业净利率'] = df3['净利润']/df3['营业收入']
df3['权益净利率'] = df3['净利润']/df3['平均所有者权益']
df3['总资产净利率'] = df3 ['净利润']/(df3['平均流动资产']+df3['平均非流动资产'])

# 创建df_2020(2020年财务指标统计)
df_2020 = df3.loc[df3['年']==2020,['年','月','营业收入','毛利率','营业净利率','权益净利率','总资产净利率']]
df_2020
```

	年	月	营业收入	毛利率	营业净利率	权益净利率	总资产净利率
12	2020	1	345700	0.3846	0.1950	0.029588	0.012658
13	2020	2	388000	0.3720	0.1910	0.031702	0.013714
14	2020	3	354780	0.3846	0.1984	0.029403	0.012913
15	2020	4	372300	0.3750	0.1927	0.029267	0.013395
16	2020	5	322000	0.4555	0.2706	0.034562	0.016008
17	2020	6	384200	0.3815	0.1959	0.029158	0.013480
18	2020	7	401200	0.3631	0.1744	0.026531	0.010587
19	2020	8	424000	0.3715	0.1865	0.029282	0.011728
20	2020	9	392000	0.3855	0.1997	0.028331	0.011453
21	2020	10	354820	0.3776	0.1945	0.024487	0.010089
22	2020	11	343500	0.3550	0.1729	0.020724	0.008573
23	2020	12	401200	0.3418	0.1560	0.021464	0.008931

运行结果

绘制折线图

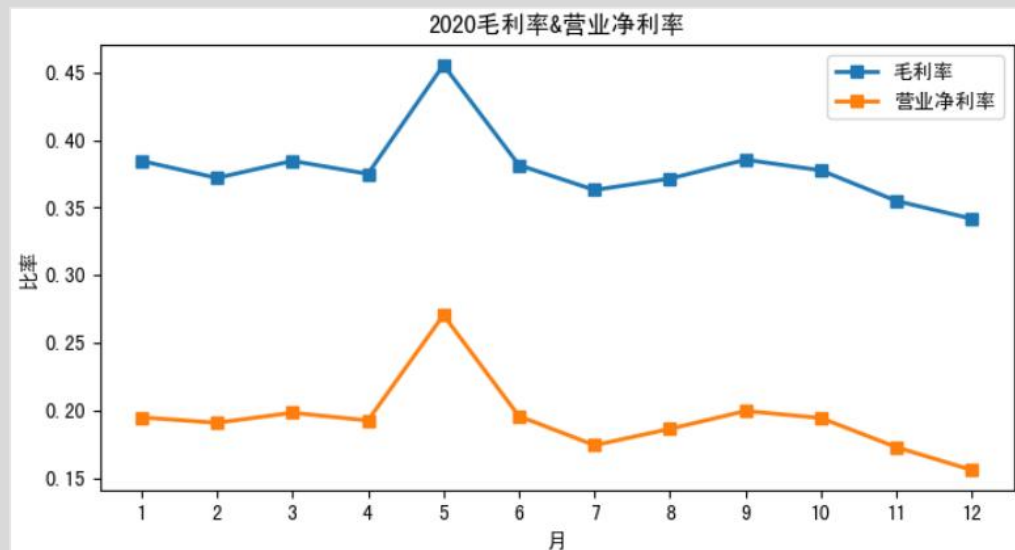
例4-1-5：根据例4-1-4中2020年“毛利率”和“营业净利率”指标绘制折线图。

```
# 创建画布
plt.figure(figsize=(8,4),dpi=100)

# 指定数据
plt.plot(df_2020['月'],df_2020['毛利率'],label='毛利率',
         linewidth=2,marker='s')
plt.plot(df_2020['月'],df_2020['营业净利率'],label='营业净利率',
         linewidth=2,marker='s')

# 设置标签、标题
plt.xlabel('月')
plt.ylabel('比率')
plt.title('2020毛利率&营业净利率')
plt.xticks(df_2020['月'])
plt.legend()

# 显示图形
plt.show()
```



运行结果

绘制柱状图

柱状图也称为条形图，是一种以长方形长度为变量的表达图形的统计报告图，由一系列高度不等的纵向条纹表示数据分布的情况，用来比较两个或者两个以上的数值。

bar()函数：绘制柱状图。语法：plt.bar(x, height, width=0.8, bottom=None, *, align='center', data=None,**kwargs)

常用参数	说明
x	x 轴数据，接收数组、列表、元组等
height	柱状的高度，即 y 轴数值，接收数组、列表、元组等
width	柱状的宽度，默认 0.8
bottom	设置 y 边界坐标轴起点
align	柱状与 x 坐标的对齐方式，默认值'center'，表示居中位置，align='edge' 表示边缘位置
data	可索引对象(如： <u>dict</u> 、 <u>DataFrame</u>)
color、 <u>edgecolor(ec)</u>	柱状填充颜色、图形边缘颜色
alpha	设置柱状的透明度，0.0-1.0 之间
label	图例内容，接收字符串

柱状图bar()函数常用参数

绘制垂直方向的柱形图可以用barh()函数，barh()函数比bar()函数名称重多了一个h，是英文单词horizontal（水平）的首字母。



绘制柱状图

例4-1-6：根据例4-1-4中2020-

df_2020['月']-0.2
df_2020['月']+0.2语句中“+”和
“-”表示左右调整位置。

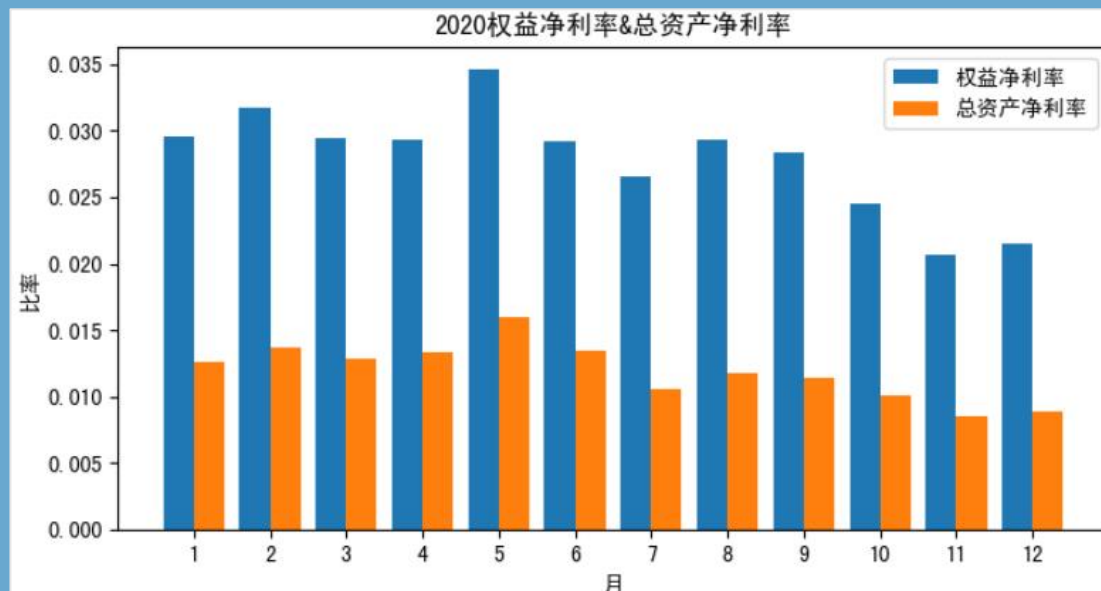
指标绘制柱状图，图形并列显示。

```
# 创建画布
plt.figure(figsize=(8,4),dpi=100)

# 指定数据
plt.bar(df_2020['月']-0.2,df_2020['权益净利率'],label='权益净利率',
        width=0.4) # x-0.2: 防止图形重合
plt.bar(df_2020['月']+0.2,df_2020['总资产净利率'],label='总资产净利率',
        width=0.4) # x+0.2: 防止图形重合

# 设置标签、标题
plt.xlabel('月')
plt.ylabel('比率')
plt.title('2020权益净利率&总资产净利率')
plt.xticks(df_2020['月'])
plt.legend()

# 显示图形
plt.show()
```



运行结果

绘制饼状图

饼图用于表示不同分类的占比情况，通过弧度大小来对比各种分类，饼图通过将一个圆饼按照分类的占比分成多个区块，整个圆饼代表数据的总量，每个区块（圆弧）表示该分类占总体比例大小。

pie()函数：绘制饼图，函数
plt.pie(x,explode=None,
abeldistance=1.1,startangle=0,
textprops=None,center=None)

常用参数	说明
x	绘图数据， <u>饼图</u> 每一部分的比例，接收数组、列表、元组，如果 sum(x)>1 则使用 sum(x) 归一化（即计算各项百分比）
explode	指定每一部分偏移中心的距离（以半径为 1，按占半径的比例设置），接收列表或元组
labels	设置标签，接收列表或元组
colors	<u>设置饼图颜色</u> ，接收列表或元组
<u>autopct</u>	设置 <u>饼图</u> 每一部分百分数格式，如：'%.2f%%'（百分数保留 2 位小数）
shadow	是否显示阴影，默认 False
radius	<u>设置饼图半径</u> ，默认值为 1
<u>labeldistance</u>	标签位置相对于半径的比例，默认值为 1.1
<u>pctdistance</u>	<u>饼图百分数</u> 显示位置相对于半径的比例，默认值为 0.6

nce=0.6,shadow=False,l
ps=None,
ze=None,data=None)

饼图pie()函数常用参数

绘制饼状图

例4-1-7：根据例4-1-4中2020年“营业收入”指标绘制饼图。

创建画布

```
plt.figure(figsize=(5,5),dpi=100)
```

指定数据

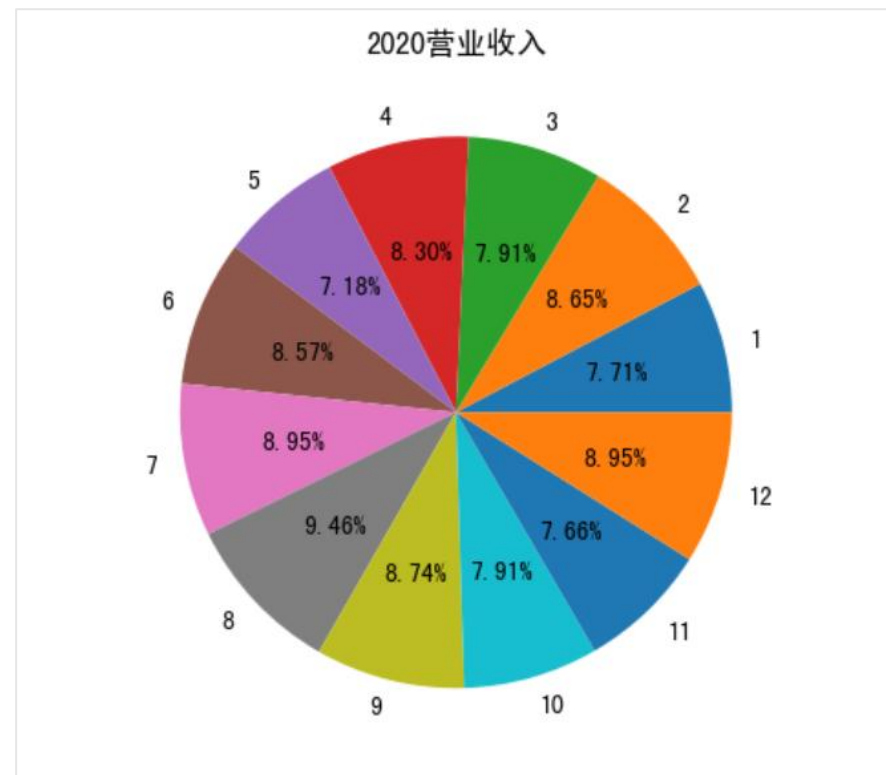
```
plt.pie(df_2020['营业收入'],labels=df_2020['月'],autopct='%.2f%%')
```

设置标题

```
plt.title('2020营业收入')
```

显示图形

```
plt.show()
```



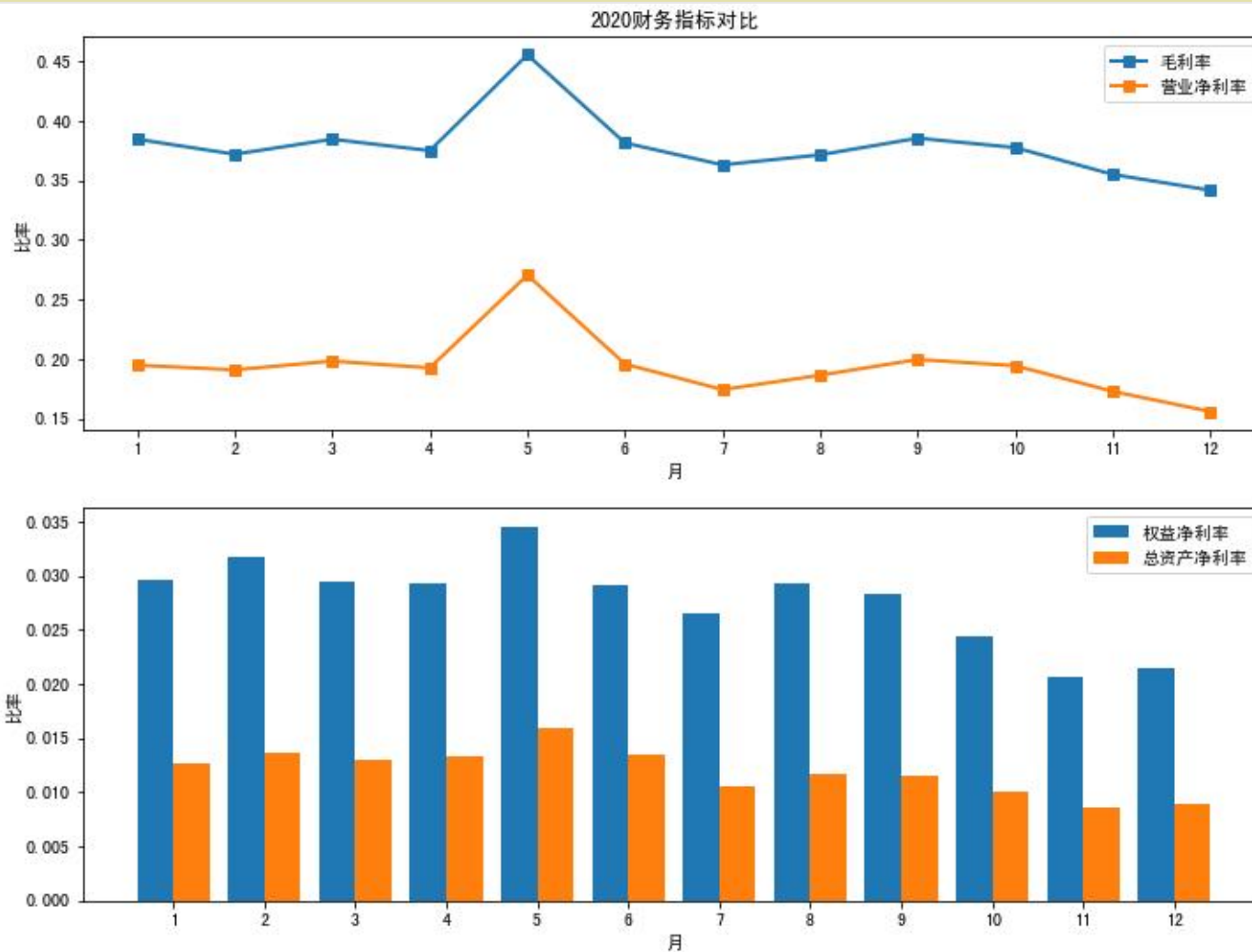
运行结果

绘制组合图形

例4-1-8：使

```
# 创建画布
plt.figure(figsize=(12, 8))

# 创建折线图绘
plt.subplot(2,1,1)
plt.plot(df_2020['毛利率'],label='毛利率',
         df_2020['营业净利率'],label='营业净利率')
plt.xlabel('月')
plt.ylabel('比率')
plt.title('2020财务指标对比')
plt.xticks(df_2020['月'])
plt.legend()
```



图组合在一起。

```
,label='权益净利率',
率'],label='总资产净
```


subplots()函数

subplots()函数：快速创建多子图环境，将画布分为n行m列的绘图空间，返回一个figure和多个axes(列表)，需要两个变量分别接收，选择区域时使用子图的列表索引进行访问，子图索引从0开始。

语法：

```
plt.subplots(nrows,ncols,sharex=False,sharey=False,squeeze=True,subplot_kw=None,gridspec_kw=None,**fig_kw)
```

nrows、ncols：子图的行数和列数，表示绘图区域被分为n行m列；

sharex、sharey：是否共享x轴或y轴，默认False代表子图的x、y轴独立；

**fig_kw：figure函数的参数都可以使用，如figsize。

subplots()函数

例4-1-9：使用subplots()函数将例4-1-5绘制的折线图和例4-1-6绘制的柱状图组合在一起。

```
# 创建子图，共享
fig,ax = plt.subplots
```

```
# 绘制折线图
```

```
ax[0].plot(df_2020
```

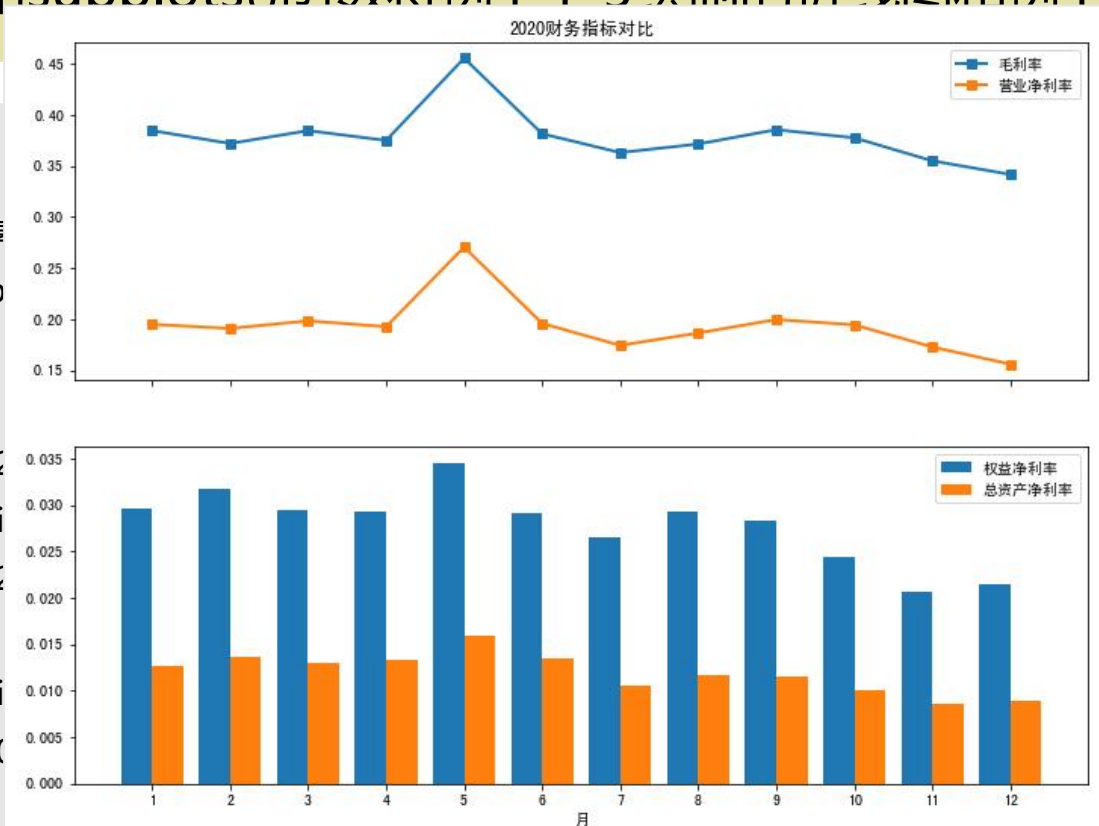
```
ax[0].plot(df_2020
```

```
率',
```

```
li
```

```
ax[0].set_title('20
```

```
ax[0].legend()
```



```
plt.show()
```

```
0.2,df_2020['权益净利率'],label='权益净利
```

```
.4)
```

```
0.2,df_2020['总资产净利率'],
```

```
dth=0.4)
```

```
['月'])
```

subplot()与subplots()



subplot()函数

需要先创建画布，再创建子图，并且每次只能返回一个坐标对象，绘图时，每次都要调用subplot指定位置。



subplots()函数

在创建画布时，会一次性建立所有子图axes，后续直接调用axes对象即可，使用该方法可以直接规划画布(指定画布大小等)。

注意

plt.subplot(2,1,1)会将原始的图像切割成2个子图像，是2行1列，并将现在的操作位置转到第一个子图上，这样便实现了绘制子图的方法。

在确定子图编号时subplot()从1开始，subplots()从0开始。

pandas作图函数-plot

Pandas提供了plot()绘图函数，可以绘制多种图形样式，相比于Matplotlib，可以直接对DataFrame调用该函数，并且函数中集成了x、y轴数据、图形标题、图例、样式等参数，代码比Matplotlib简洁许多。



注意

使用Pandas作图内部依赖于Matplotlib，因此在使用plot()之前，必须引入Matplotlib库。

plot()函数语法：

```
DataFrame.plot(x=None,y=None,kind='line',ax=None,subplots=False,sharex=None,sharey=False,layout=None,figsize=None,use_index=True,title=None,grid=None,legend=True,style=None,logx=False,logy=False,loglog=False,xticks=None,yticks=None,xlim=None,ylim=None,rot=None,secondary_y=False,sort_columns=False,**kwargs)
```

pandas作图函数-plot

参数	说明
x	指定 x 轴上显示的数据列，默认使用行索引
y	指定 y 轴上显示的数据列，默认所有数值型数据列
kind	绘图类型，默认为折线图“line”，可选参数有：“ <u>barh</u> ”（水平条形图）、“hist”（直方图）、“ <u>kde</u> ”（密度估计图）、“area”（面积图）、“scatter”（散点图）、“ <u>hexbin</u> ”（蜂巢图）
ax	选择子绘图区域，默认 None
subplots	<u>是否按列绘制子图</u> ，默认 False
<u>sharex</u>	共享 x 轴，如果 ax=None 则默认为 True，否则为 Fa
<u>sharey</u>	共享 y 轴，默认为 False
layout	子图的行列布局，元组形式(行, 列)
<u>figsize</u>	图形尺寸大小，元组形式(宽度, 高度)
<u>use_index</u>	用索引做 x 轴，默认为 True
title	图形的标题，字符串或列表，若传递字符串则在图顶部打印该字符串，若传递列表且含有子图，则依次在每个子图顶部打印标题

grid	图表是否有网格，默认 None
legend	子图的图例显示，默认为 True，可选‘reverse’，颠倒图例顺序
style	对每列折线图设置线的样式，列表或字典
<u>logx</u>	设置 x 轴刻度是否取对数，默认 False
logy	设置 y 轴刻度是否取对数，默认 False
loglog	同时设置 x、y 轴刻度是否取对数，默认 False
<u>xticks</u>	设置 x 轴刻度，使用序列形式（如列表）
<u>yticks</u>	设置 y 轴刻度，使用序列形式（如列表）
<u>xlim</u>	设置 x 轴的数值范围，使用列表或元组形式
<u>ylim</u>	设置 y 轴的数值范围，使用列表或元组形式
rot	设置轴标签(轴刻度)的显示旋转度数，默认 None
<u>secondary_y</u>	设置第二个 y 轴（右 y 轴），默认 False，也可以列表或元组形式指定列
<u>sort_columns</u>	对列名称进行排序以确定绘图顺序，默认为 False <u>不排序</u>
<u>**kwargs</u>	在 matplotlib 绘图方法中相关参数都可以在此函数中使用，如 color、 <u>linestyle</u> 、linewidth、marker 等

plot()绘图函数常用参数

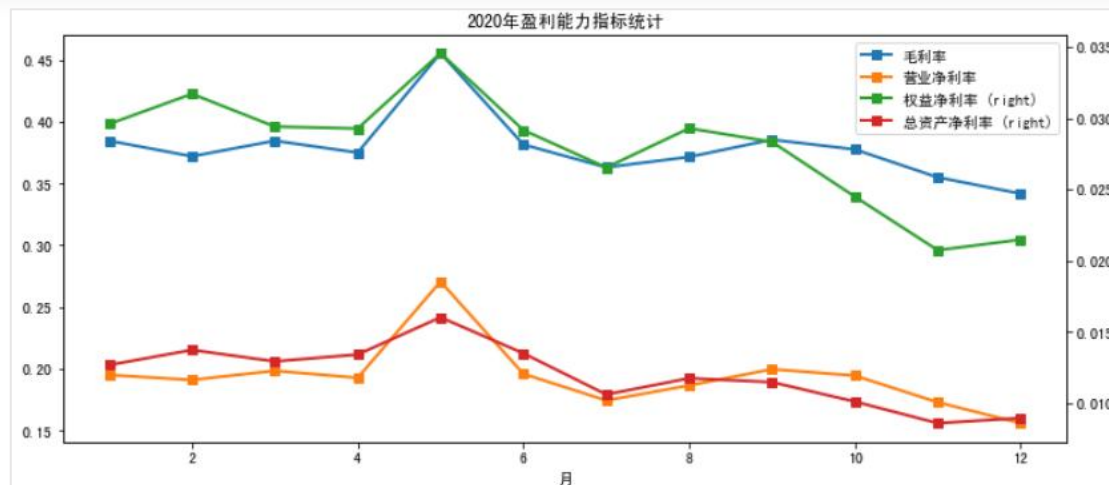
pandas作图函数-plot

例4-1-10：根据例4-1-4中2020年毛利率、营业净利率、权益净利率、总资产净利率指标进行可视化展示，绘制折线图，并针对权益净利率、总资产净利率指标设置右y轴。

折线图

```
df_2020.plot('月',['毛利率','营业净利率','权益净利率','总资产净利率'],  
             secondary_y=['权益净利率','总资产净利率'],linewidth=2,  
             marker='s',title='2020年盈利能力指标统计',figsize=(12,5));
```

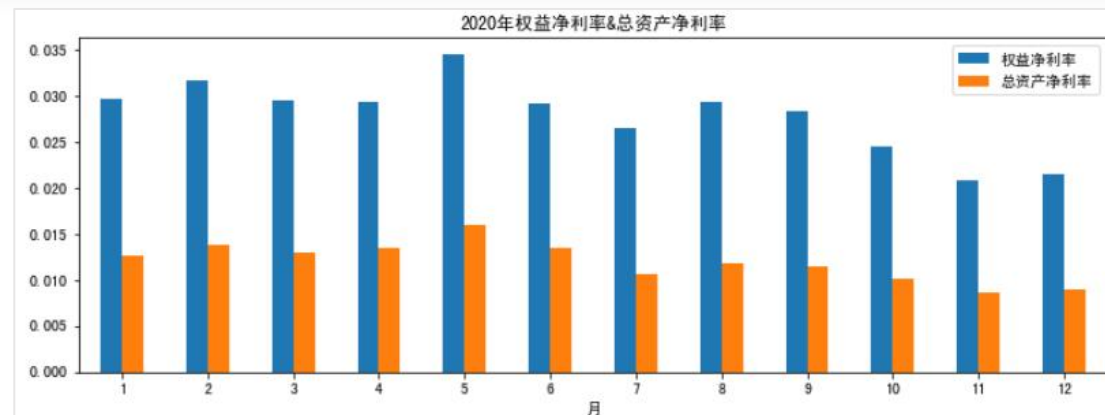
运行结果



pandas作图函数-plot

例4-1-11：根据例4-1-4中2020年权益净利率、总资产净利率指标绘制柱状图。

```
# 柱状图
df_2020.plot('月',['权益净利率','总资产净利率'],kind='bar',
title='2020年权益净利率&总资产净利率',
figsize=(12,4),rot=0);
```



运行结果

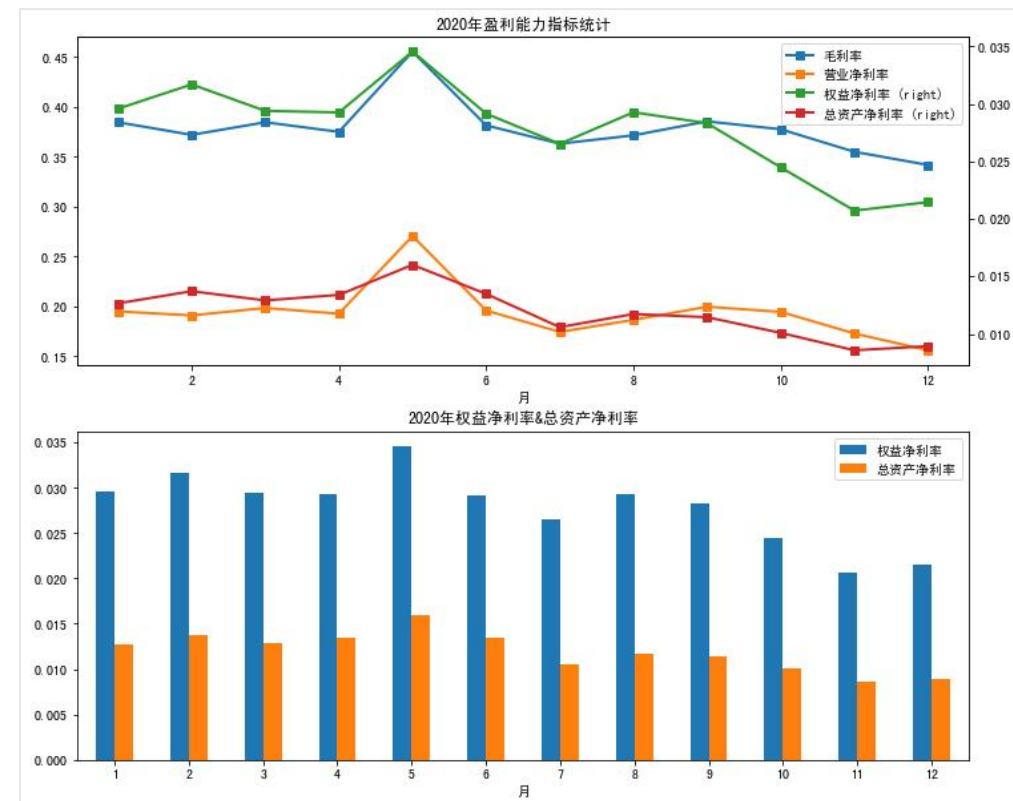
pandas作图函数-plot

例4-1-12：根据例4-1-10和例4-1-11可视化结果，绘制组合图形。

```
# 创建子图
fig,ax = plt.subplots(2,1,figsize=(12,10))

# 绘制 “2020年盈利能力指标统计”
df_2020.plot('月',['毛利率','营业净利率','权益净利率','总资产净利率'],
             secondary_y=['权益净利率','总资产净利率'],linewidth=2,
             marker='s',title='2020年盈利能力指标统计',ax=ax[0])

# 绘制 “2020年权益净利率&总资产净利率”
df_2020.plot('月',['权益净利率','总资产净利率'],
             kind='bar',ax=ax[1],rot=0,
             title='2020年权益净利率&总资产净利率');
```



运行结果

小结

matplotlib绘图流程：创建画布→选定子图→绘制图形→添加图例→保存图形→显示图形

figure：创建空白画布，一个画布中包含一个或多个坐标系(Axes)，每个Axes都是一个绘图区域

subplot：创建子图并选择绘图区域

subplots：快速创建多子图环境，需要两个变量分别接收

其他常用函数：title、label、tick、legend等

常用绘图函数：plot、bar、pie等

作图函数-plot：Pandas快速作图函数，通过参数可绘制各种图形

matplotlib官网：<https://matplotlib.org>



02

财务数据可视化-Pyecharts

Pyecharts简介

Echarts(Enterprise Charts, 即商业级数据图表), 是一个由百度开源的数据可视化库, 凭借着良好的交互性、精巧的图表设计, 得到了众多开发者的认可。Echarts除了支持常规的折线图、柱状图、饼图等基本图形外, 还支持树形图、地理图、3D图以及组合图形。

Pyecharts支持图表类型有很多种, 包括基本图表、直角坐标系图表、树型图表、地理图表、3D图表、组合图表等。

安装Pyecharts库, 执行步骤如下:

第一步: 打开“命令提示符”(快捷键win+R)

第二步: 在命令提示符中输入代码: `pip3 install Pyecharts`

Pyecharts初级应用

Pyecharts图表类引入: `from pyecharts.charts import 图表类名`

使用`add()`方法添加数据及配置项

使用`render_notebook()`在notebook中展示图形

01

初始化具体类型图表

02

03

使用`render()`生成图形(.html文件)

04

Pyecharts初级应用

例4-2-1：甲公司2020年1-5月份收入金额依次为45000元、38000元、54000元、55000元、48000元，成本金额依次为31500元、28500元、38880元、40700元、35520元。使用Pyecharts绘制1-5月收入成本对比图(图表类型为柱状图)。

```
# 初始化图表类型
from pyecharts.charts import Bar
bar1 = Bar()

# 添加数据
bar1.add_xaxis(['1月','2月','3月','4月','5月'])
bar1.add_yaxis('收入',[45000,38000,54000,55000,48000])
bar1.add_yaxis('成本',[31500,28500,38880,40700,35520])

# 生成html文件
bar1.render('收入&成本对比.html')

# 展示图形
bar1.render_notebook()
```

Pyecharts还支持链式调用：

```
from pyecharts.charts import Bar
bar1 = (Bar()
        .add_xaxis(['1月','2月','3月','4月','5月'])
        .add_yaxis('收入',[45000,38000,54000,55000,48000])
        .add_yaxis('成本',[31500,28500,38880,40700,35520])
        )

# 展示图形
bar1.render_notebook()
```

Pyecharts高级应用

Pyecharts提供了丰富的配置项，包括全局配置项和系列配置项：

`set_global_opts()`：全局配置项，可配置标题、图例、坐标轴、工具箱等；

`set_series_opts()`：系列配置项，可配置图元样式、文字样式、标签样式、点线样式等。

Pyecharts配置项引入方法：

```
from pyecharts import options as opts
```

初始化配置

初始化配置: `init_opts = opts.InitOpts()`

常用参数	说明
<code>width</code>	str, 图表画布宽度(像素 <u>px</u>), 如: <code>width=‘900px’</code>
<code>height</code>	str, 图表画布高度(像素 <u>px</u>), 如: <code>height=‘500px’</code>
<u><code>chart_id</code></u>	str, 图表 ID, 图表唯一标识
<code>theme</code>	图表主题

`opts.InitOpts()`常用参数

定制主题

主题类型引入方法: `from pyecharts.globals import ThemeType`

设置主题方法: `theme = ThemeType.主题风格`

BUILTIN_THEMES	INFOGRAPHIC	ROMA	WALDEN
CHALK	LIGHT	ROMANTIC	WESTEROS
DARK	MACARONS	SHINE	WHITE
ESSOS	PURPLE_PASSION	VINTAGE	WONDERLAND

pyecharts内置主题

定制主题

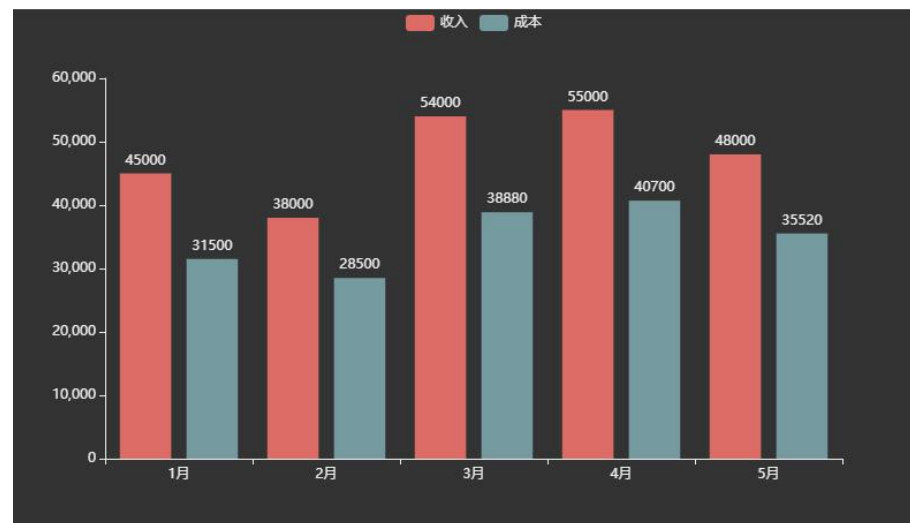
例4-2-2：在例4-2-1图形的基础上，配置宽度为800Px,高度为450px，主题为“DARK”。

```
# 引入表格类型、配置项、主题类型
from pyecharts.charts import Bar
from pyecharts import options as opts
from pyecharts.globals import ThemeType

# 初始化配置，设置主题风格
bar1 =
Bar(init_opts=opts.InitOpts(width='800px',height='450px',
theme=ThemeType.DARK))

# 添加数据
bar1.add_xaxis(['1月','2月','3月','4月','5月'])
bar1.add_yaxis('收入',[45000,38000,54000,55000,48000])
bar1.add_yaxis('成本',[31500,28500,38880,40700,35520])

# 展示图形
bar1.render_notebook()
```



运行结果

常用全局配置项

调用标题配置项: `title_opts = opts.TitleOpts()`

常用参数	说明
<code>title</code>	主标题文本, 支持使用\n 换行
<code>subtitle</code>	副标题文本, 支持使用\n 换行
<code>pos_left</code>	标题组件离容器左侧的距离, 可以是具体数值, 也可以是百分比, 还可以是'left', 'center', 'right', 组件会根据相应的位置自动对齐
<code>pos_right</code>	标题组件离容器右侧的距离
<code>pos_top</code>	标题组件离容器上侧的距离, 可以是具体数值, 也可以是百分比, 还可以是'top', 'middle', 'bottom', 组件会根据相应的位置自动对齐
<code>pos_bottom</code>	标题组件离容器下侧的距离

标题配置项`opts.TitleOpts()`常用参数

注意

`TitleOpts ()` 函数的各个参数要以 “, ” 分割, 可以加入适当换行符和注释方便阅读, 最后 “)” 前不要有 “, ” 。

常用全局配置项

例4-2-3：在例4-2-2图形的基础上，设置主标题为“收入&成本对比”，副标题为“2020年1-5月”。

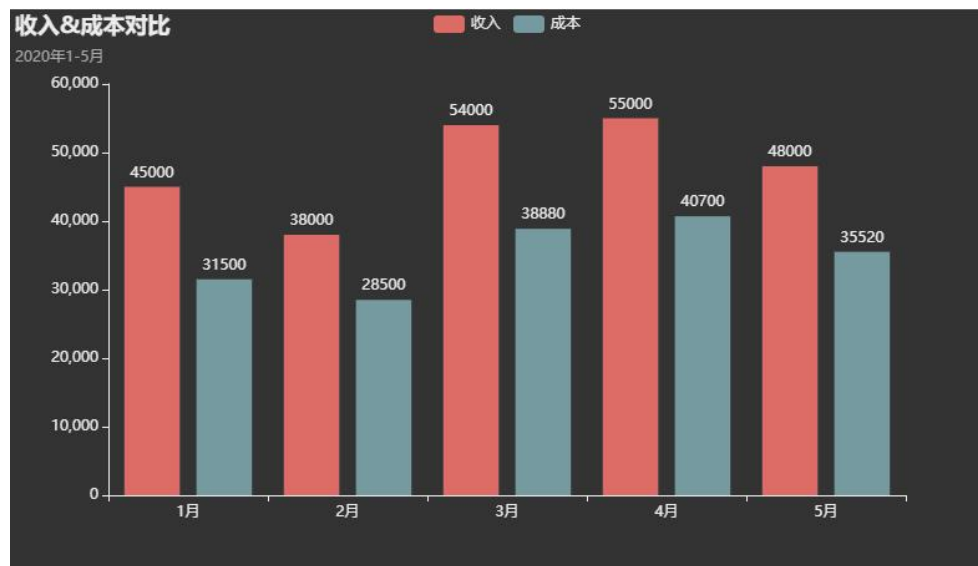
```
# 引入表格类型、配置项、主题类型
from pyecharts.charts import Bar
from pyecharts import options as opts
from pyecharts.globals import ThemeType

# 初始化配置
bar1 =
Bar(init_opts=opts.InitOpts(width='800px',height='450px',
                             theme=ThemeType.DARK))

# 添加数据
bar1.add_xaxis(['1月','2月','3月','4月','5月'])
bar1.add_yaxis('收入',[45000,38000,54000,55000,48000])
bar1.add_yaxis('成本',[31500,28500,38880,40700,35520])

# 设置全局配置项
bar1.set_global_opts(title_opts = opts.TitleOpts(title=
'收入&成本对比',subtitle='2020年1-5月'))

# 展示图形
bar1.render_notebook()
```



运行结果

图例配置项

调用图例配置项: `legend_opts = opts.LegendOpts()`

常用参数	说明
<code>type_</code>	图例的类型, 可选, 默认'plain': 代表普通图例, 'scroll': 代表可滚动翻页的图例。
<code>selected mode</code>	图例选择的模式, 控制是否可以通过点击图例改变系列的显示状态。默认开启图例选择, 可以设成 False 关闭, 也可设成'single' 或者'multiple' 使用 <u>单选或者多选</u> 模式。
<code>is show</code>	是否显示图例组件, 默认为 True
<code>pos_left/pos_right</code>	图例组件离容器左/右侧的距离
<code>pos_top/pos_bottom</code>	图例组件离容器上/下侧的距离
<code>orient</code>	图例列表的布局朝向, 默认'horizontal' 水平, 可选: 'vertical' 垂直

图例配置项LegendOpts()常用参数

图例配置项

例4-2-4：在例4-2-3图形的基础上，将图例垂直排布。

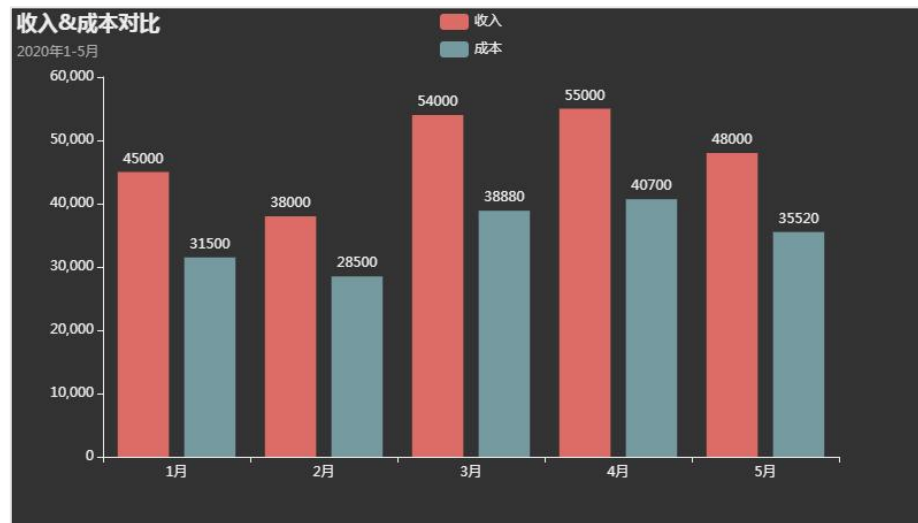
```
# 引入表格类型、配置项、主题类型
from pyecharts.charts import Bar
from pyecharts import options as opts
from pyecharts.globals import ThemeType

# 初始化配置
bar1 =
Bar(init_opts=opts.InitOpts(width='800px',height='450px',
                             theme=ThemeType.DARK))

# 添加数据
bar1.add_xaxis(['1月','2月','3月','4月','5月'])
bar1.add_yaxis('收入',[45000,38000,54000,55000,48000])
bar1.add_yaxis('成本',[31500,28500,38880,40700,35520])

# 设置全局配置项
bar1.set_global_opts(title_opts = opts.TitleOpts(title=
'收入&成本对比',subtitle='2020年1-5月'),
                     legend_opts =
opts.LegendOpts(orient='vertical'))

# 展示图形
bar1.render_notebook()
```



运行结果

坐标轴配置项

调用x、y轴配置项: `xaxis_opts = opts.AxisOpts()`
`yaxis_opts = opts.AxisOpts()`

常用参数	说明
<code>name</code>	坐标轴名称
<code>is_show</code>	是否显示坐标轴, 默认为 True
<code>is_inverse</code>	是否反向坐标轴, 默认为 False
<code>name_location</code>	坐标轴名称显示位置, 可选: <code>'start', 'middle', 'center', 'end'</code> , 默认 <code>'end'</code>
<code>name_gap</code>	坐标轴名称与轴线之间的距离, 默认 15
<code>name_rotate</code>	坐标轴名称旋转角度值, 解决坐标轴名称过长的问题
<code>axislabel_opts</code>	坐标轴标签配置项。如: 设置坐标轴标签旋转角度值, 解决坐标轴标签名称过长的问题: <code>axislabel_opts=opts.LabelOpts(rotate=-30)</code>

坐标轴配置项AxisOpts()常用参数

坐标轴配置项

例4-2-5：在例4-2-4图形的基础上，分别设置x轴和y轴标签为“月份”、“金额（元）”，并根据图形调整标签位置。

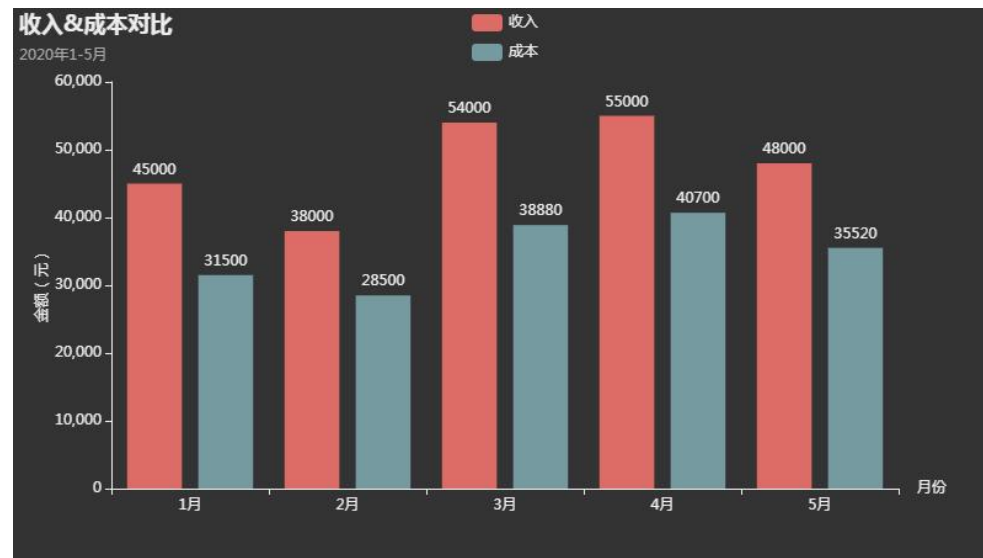
```
# 引入表格类型、配置项、主题类型
from pyecharts.charts import Bar
from pyecharts import options as opts
from pyecharts.globals import ThemeType

# 初始化配置
bar1 =
Bar(init_opts=opts.InitOpts(width='800px',height='450px',
                             theme=ThemeType.DARK))

# 添加数据
bar1.add_xaxis(['1月','2月','3月','4月','5月'])
bar1.add_yaxis('收入',[45000,38000,54000,55000,48000])
bar1.add_yaxis('成本',[31500,28500,38880,40700,35520])

# 设置全局配置项
bar1.set_global_opts(title_opts = opts.TitleOpts(title=
'收入&成本对比',subtitle='2020年1-5月'),
                    legend_opts=opts.LegendOpts(orient='vertical'),
                    axis_opts=opts.AxisOpts(name='月份'),
                    yaxis_opts=opts.AxisOpts(name='金额（元）',
                    name_location='center',
                    name_gap=50))

# 展示图形
bar1.render_notebook()
```



收入&成本对比

工具箱配置

调用工具箱配置: `toolbox_opts = opts.ToolboxOpts()`

常用参数	说明
<u>is_show</u>	是否显示工具栏组件, 默认 False
orient	工具栏的布局朝向, 可选: 默认 'horizontal' 水平, 'vertical' 垂直
<u>pos_left</u>	图例组件离容器左侧的距离
<u>pos_right</u>	图例组件离容器右侧的距离
<u>pos_top</u>	图例组件离容器上侧的距离
<u>pos_bottom</u>	图例组件离容器下侧的距离

工具箱配置ToolboxOpts()常用参数

工具箱配置

例4-2-6：在例4-2-5图形的基础上，添加一个工具箱。

```
# 引入表格类型、配置项、主题类型
```

```
from pyecharts.charts import Bar
```

```
from pyecharts import options as opts
```

```
from pyecharts.globals import Theme
```

```
# 初始化配置
```

```
bar1 =
```

```
Bar(init_opts=opts.InitOpts(
```

```
theme=Theme
```

```
# 添加数据
```

```
bar1.add_xaxis(['1月','2月','3月','4月','5月'])
```

```
bar1.add_yaxis('收入',[45000,38000,54000,55000,48000])
```

```
bar1.add_yaxis('成本',[31500,28500,38880,40700,35520])
```

```
# 设置全局配置项
```

```
opts = opts.TitleOpts(title=
```

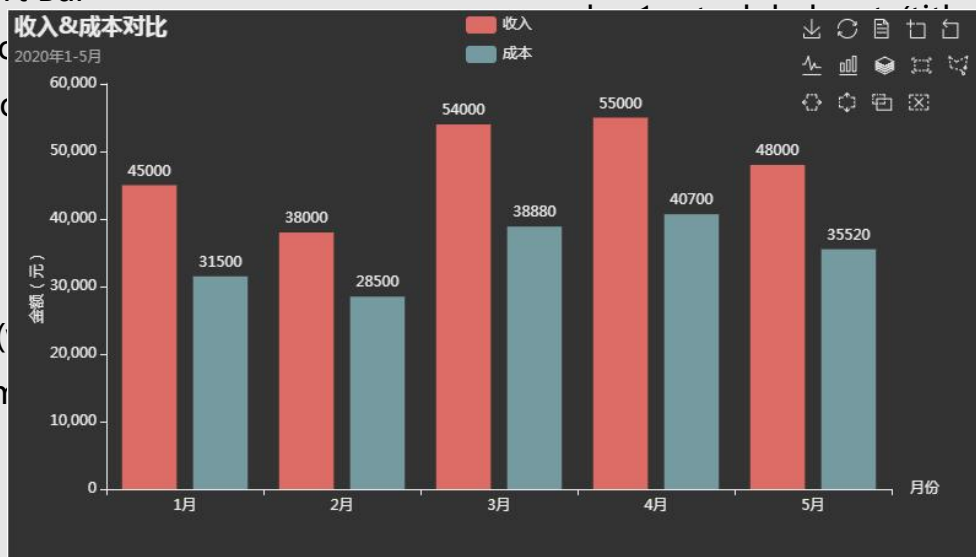
```
'2020年1-5月'),
```

```
opts.LegendOpts(orient='vertical'),
```

```
opts.AxisOpts(name='月份'),
```

```
opts.AxisOpts(name='金额 (元) ',
```

```
opts.ToolboxOpts(is_show=True))
```



```
# 展示图形
```

```
bar1.render_notebook()
```


常用系列配置项

例4-2-6：在例4-2-5图形的基础上，添加一个工具箱。

常用系列配置项	说明
标签配置项	调用配置： <u>label_opts=opts.LabelOpts()</u> ，可设置标签的字体、大小、位置、旋转角度等， 常用参数： <u>is_show</u> (是否显示标签)
线样式配置项	调用配置： <u>linestyle_opts=opts.LineStyleOpts()</u> ，可设置线条的宽度、透明度、颜色等， 常用参数： <u>width</u> (调整线条宽度)
区域填充样式配置项	调用配置： <u>areastyle_opts=opts.AreaStyleOpts()</u> ，可设置区域填充的透明度、颜色等， 常用参数： <u>opacity</u> (设置透明度，0-1 的数字，可通过该参数绘制面积图)

配置项set_series_opts()函数常用配置

DataFrame数据可视化-数据格式转换

Pyecharts 本质上是将 Echarts 的配置项由 Python dict 序列化为 JSON 格式，所以 Pyecharts 支持什么格式的数据类型取决于 JSON 支持什么数据类型，Python 中对 JSON 的格式转换如表所示：

Python	JSON
int, float	number
str	string
bool	<u>boolean</u>
<u>dict</u>	object (JSON 对象)
list	array

Python对JSON的格式转换表

将数据传入 Pyecharts 时，需要先将数据格式转换成上述Python原生的数据类型才可使用；而数据分析一般需要使用 Numpy、Pandas等第三方库，Numpy中的int64、int32等数据类型并不继承自 Python原生数据类型，所以在这些数据时需要将其转换为Python原始数据类型。

转换方式：Series.tolist()

数据准备

例4-2-7：读取data.xlsx文件，根据“资产负债表项目”和“利润表项目”计算各期毛利率、营业净利率、权益净利率、总资产净利率，并计算各期毛利率、营业净利率、权益净利率、总资产净利率，并筛选2020年相关指标。
URL地址：<https://keyun-oss.acctedu.com/app/bigdata/basics/data.xlsx>

```
# 引入pandas
import pandas as pd
```

```
# 读取资产负债表项目
df1 = pd.read_excel(r'https://keyun-oss.acctedu.com/app/bigdata/basics/data.xlsx',sheet1,
                    converters={'年':lambda x: int(x)-2020})
```

```
# 读取利润表项目
df2 = pd.read_excel(r'https://keyun-oss.acctedu.com/app/bigdata/basics/data.xlsx',sheet2,
                    converters={'年':lambda x: int(x)-2020})
```

```
# 调用merge函数连接df1、df2
df3 = df1.merge(df2, on='年', how='left')
```

	年	月	营业收入	毛利率	营业净利率	权益净利率	总资产净利率
0	2020	1	345700	0.3846	0.1950	0.029588	0.012658
1	2020	2	388000	0.3720	0.1910	0.031702	0.013714
2	2020	3	354780	0.3846	0.1984	0.029403	0.012913
3	2020	4	372300	0.3750	0.1927	0.029267	0.013395
4	2020	5	322000	0.4555	0.2706	0.034562	0.016008
5	2020	6	384200	0.3815	0.1959	0.029158	0.013480
6	2020	7	401200	0.3631	0.1744	0.026531	0.010587
7	2020	8	424000	0.3715	0.1865	0.029282	0.011728
8	2020	9	392000	0.3855	0.1997	0.028331	0.011453
9	2020	10	354820	0.3776	0.1945	0.024487	0.010089
10	2020	11	343500	0.3550	0.1729	0.020724	0.008573
11	2020	12	401200	0.3418	0.1560	0.021464	0.008931

```
df3['毛利率'] = (df3['营业收入'] - df3['营业成本']) / df3['营业收入']
df3['营业净利率'] = df3['净利润'] / df3['营业收入']
df3['权益净利率'] = df3['净利润'] / df3['平均所有者权益']
df3['总资产净利率'] = df3['净利润'] / (df3['平均流动资产'] + df3['平均流动负债'])
df3['财务指标统计'] = df3[['年', '月', '营业收入', '毛利率', '营业净利率', '权益净利率', '总资产净利率']]
```

df3_2020

绘制饼图

例4-2-8：在例4-2-7的基础上，使用Pyecharts根据2020年营业收入指标绘制饼图，要求展示玫瑰饼图。

```
from pyecharts.charts import Pie
from pyecharts import options as opts

# 按营业收入降序排序
df_sorted = df_2020.sort_values('营业收入', ascending=False)

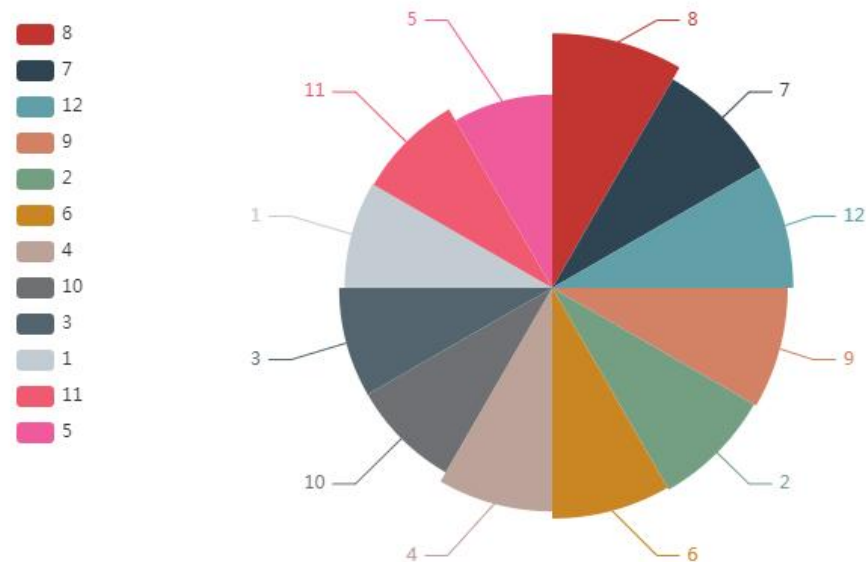
# 数据转换：将DataFrame转换为Python数据类型列表
x = df_sorted['月'].tolist()
y1 = df_sorted['营业收入'].tolist()

# 系列数据项，格式为 [(key1, value1), (key2, value2)]
data = [z for z in zip(x, y1)]

# 绘制饼图
pie = Pie()
pie.add('营业收入', data, rosetype='area')
pie.set_global_opts(title_opts=opts.TitleOpts(title='2020营业收入'),
                    legend_opts =
opts.LegendOpts(pos_left='5%', pos_top='10%',

orient='vertical'))
pie.render_notebook()
```

2020营业收入



绘制面积图

例4-2-9：在例4-2-7的基础上，使用Pyecharts根据2020年毛利率、营业净利率、权益净利率、总资产净利率指标绘制面积图。

```
# 引入表格类型、配置项、主题类型
from pyecharts.charts import Line
from pyecharts import options as opts
from pyecharts.globals import ThemeType
```

```
# 数据转换：将DataFrame转换为Python数据类型列表
```

```
x = df_2020['月'].tolist()
y2 = df_2020['毛利率'].tolist()
y3 = df_2020['营业净利率'].tolist()
y4 = df_2020['权益净利率'].tolist()
y5 = df_2020['总资产净利率'].tolist()
```

```
# 初始化配置
```

```
line1 =
Line(init_opts=opts.InitOpts(wi
    theme=Theme
```

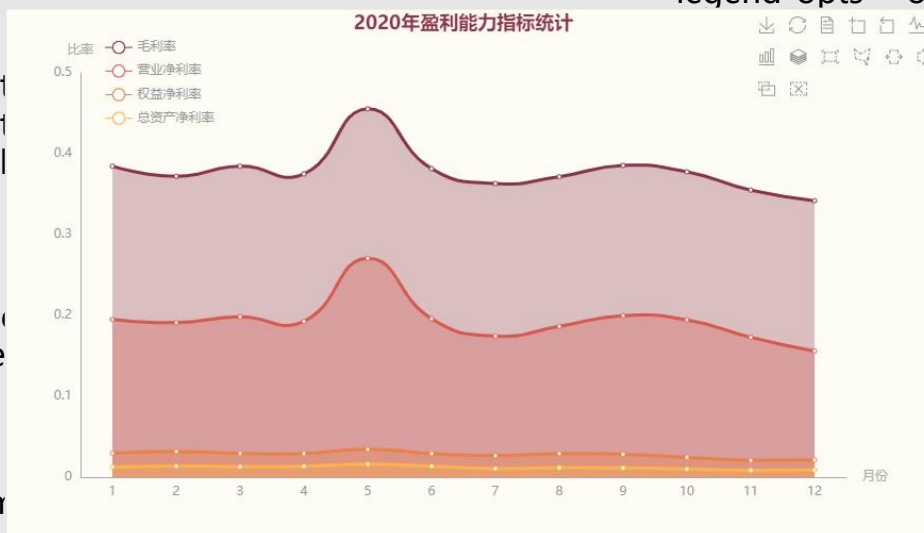
```
# 添加数据
```

```
line1.add_xaxis(x)
line1.add_yaxis('毛利率',y2,is_s
line1.add_yaxis('营业净利率',y3,is_smooth=True)
line1.add_yaxis('权益净利率',y4,is_smooth=True)
line1.add_yaxis('总资产净利率',y5,is_smooth=True)
```

```
# 设置全局配置项
```

```
line1.set_global_opts(title_opts = opts.TitleOpts(title='2020
年盈利能力指标统计',pos_left='center'),
```

```
legend_opts = opts.LegendOpts(pos_left='12%',
```



```
ical'),
AxisOpts(name='月份'),
AxisOpts(name='比率'),
ts.ToolboxOpts(is_show=True))
```

```
style_opts=opts.LineStyleOpts(width
```

```
s(is_show=False),
```

```
StyleOpts(opacity=0.3))
```

```
# 设置图形填充透明度
```

```
line1.render_notebook()
```

Pyecharts组合图形

Pyecharts 也可以将多个图形组合在一起，常用组合方法如下：

Grid：并行多图，在同一画布中显示多张图；

page：顺序多图，在同一网页中按顺序展示多图，常应用于绘制大数据看板；

Tab：选项卡多图，可以通过点击选项卡查看不同图形；

Timeline：时间线轮播多图，可以动态展示数据变化

引入方法：from pyecharts.charts import Grid,Page,Tab,Timeline

组合图形 绘图流程

01 引入
Grid,page,Tab,Ti
meline组合方法
及具体图表类型

02
绘制单个图形

03
初始化组合图形

04
使用add()添加需
要展示的图形，并
设置显示位置

05
展示组合图形

并行多图-Grid

Grid常用配置项为直角坐标系网格配置项: `grid_opts = opts.GridOpts()`

常用参数	说明
<code>is_show</code>	是否显示直角坐标系网格, 默认 False
<code>pos_left</code>	grid 组件离容器左侧的距离
<code>pos_right</code>	grid 组件离容器右侧的距离
<code>pos_top</code>	grid 组件离容器上侧的距离
<code>pos_bottom</code>	grid 组件离容器下侧的距离

并行多图GridOpts()常用参数

注意

使用Grid组合图形, 第一个添加的图形必须为直角坐标系图表, 即有 x/y 轴的图形(如: 折线图、柱状图), 其他位置顺序任意。

并行多图-Grid

例4-2-10：在例4-2-7的基础上，使用Pyecharts绘制毛利率折线图、权益净利率&总资产净利率柱状图，并使用Grid将两个图形组合在一起(上下组合)。

```
# 引入Grid
```

```
from pyecharts.charts import Grid
```

```
# 绘制毛利率折线图
```

```
line2 = Line()
```

```
line2.add_xaxis(x)
```

```
line2.add_yaxis('毛利率',y2)
```

```
line2.set_global_opts(title_opts=opts.TitleOpts(
```

```
legend_opts=opts.LegendO
```

```
line2.set_series_opts(label_o  
se))
```

```
# 绘制权益净利率&总资产净利率柱状图
```

```
bar2 = Bar()
```

```
bar2.add_xaxis(x)
```

```
bar2.add_yaxis('权益净利率',y4)
```

```
bar2.add_yaxis('总资产净利率',y5)
```

```
bar2.set_global_opts(title_opts=opts.TitleOpts(
```

```
总资产净利率',
```

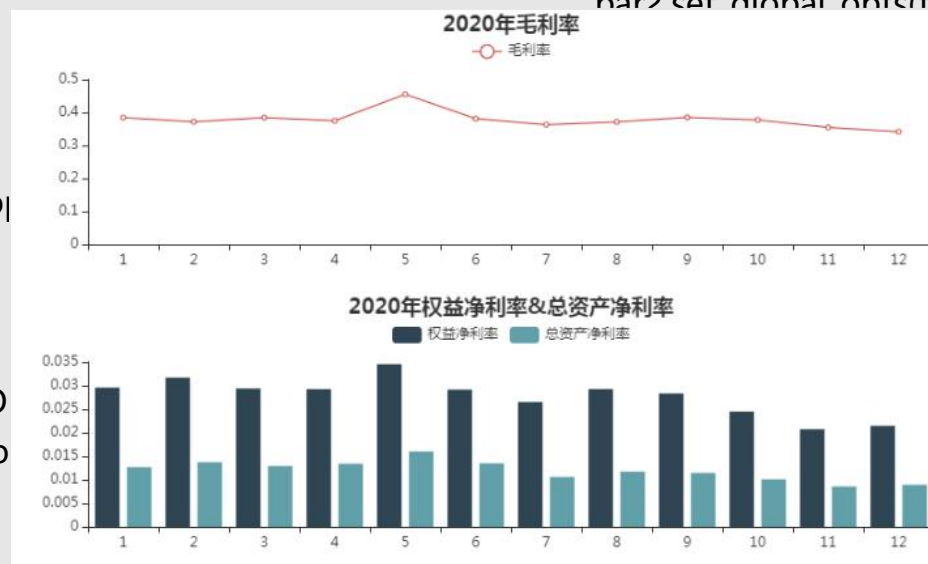
```
='48%'),
```

```
Opts(pos_top='53%'))
```

```
opts=opts.LabelOpts(is_show=Fa
```

```
opts.GridOpts(pos_bottom='60%
```

```
opts.GridOpts(pos_top='60%'))
```



```
# 展示组合图形
```

```
grid1.render_notebook()
```


顺序多图-Page

Page()常用参数: layout(布局配置项), 即页面布局。

layout = Page.SimplePageLayout # 简单页面布局, 默认选项

layout = Page.DraggablePageLayout # 可拖动页面布局, 子图可调节位置

例4-2-11: 将例4-2-8、例4-2-9、例4-2-10绘制的图形, 使用Page()函数组合在一起, 并设置可拖动页面布局。

引入Page

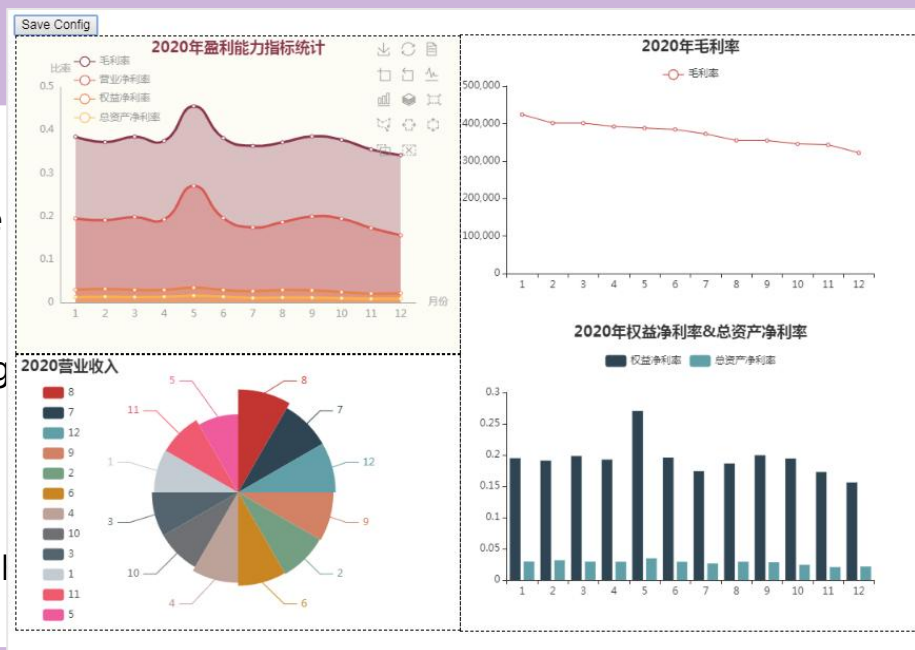
```
from pyecharts.charts import Page
```

组合图形

```
page1 = Page(layout = Page.DruggablePageLayout)
page1.add(line1,grid1,pie)
```

生成文件

```
page1.render('2020年盈利能力指标.html')
```

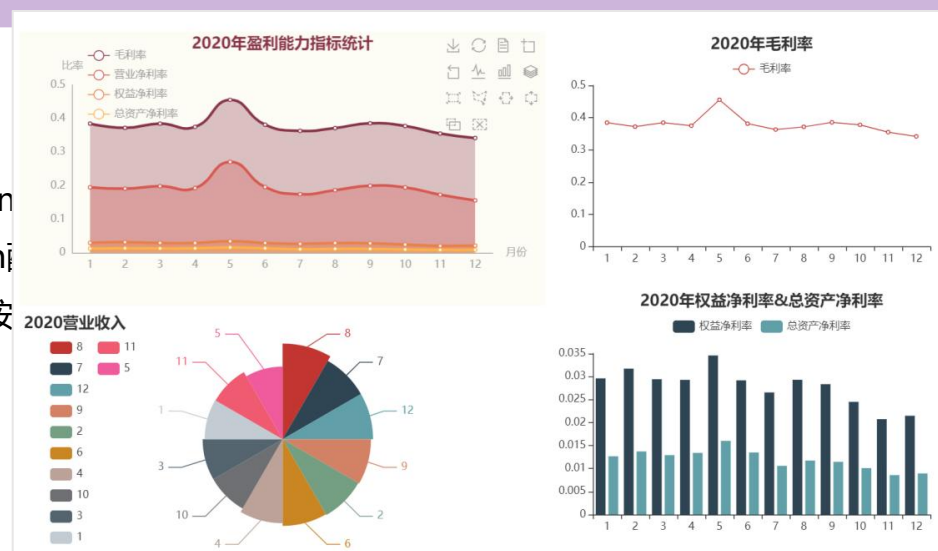


指标.html", 根据可任意缩放大小), 调节如图4-2-11所示:

顺序多图-Page

例4-2-11：将例4-2-8、例4-2-9、例4-2-10绘制的图形，使用Page()函数组合在一起，并设置可拖动页面布局。

点击“Save Con
一个chart_config.json
至jupyter文件夹后，按



ml文件
e_html('2020年盈利能力指标.html',
cfg_file='chart_config.json',
dest='2020年盈利能力指标2.html');

顺序多图-Page

注意1

部分浏览器点击“Save Config”无法下载json配置文件，这时可以先将“2020年盈利能力指标.html”下载到本地计算机，再使用浏览器打开文件即可进行调整。

注意2

渲染配置json数据中是以chart_id来作为一个图形的唯一标识符，若未指定每个图形的chart_id，则重复运行代码时，默认的chart_id将发生变化，所以，如果需要复用调整好的渲染模板，则需要指定各图形的chart_id，且需要注意的是chart_id不可与render_notebook()同时使用。

选项卡多图-Tab

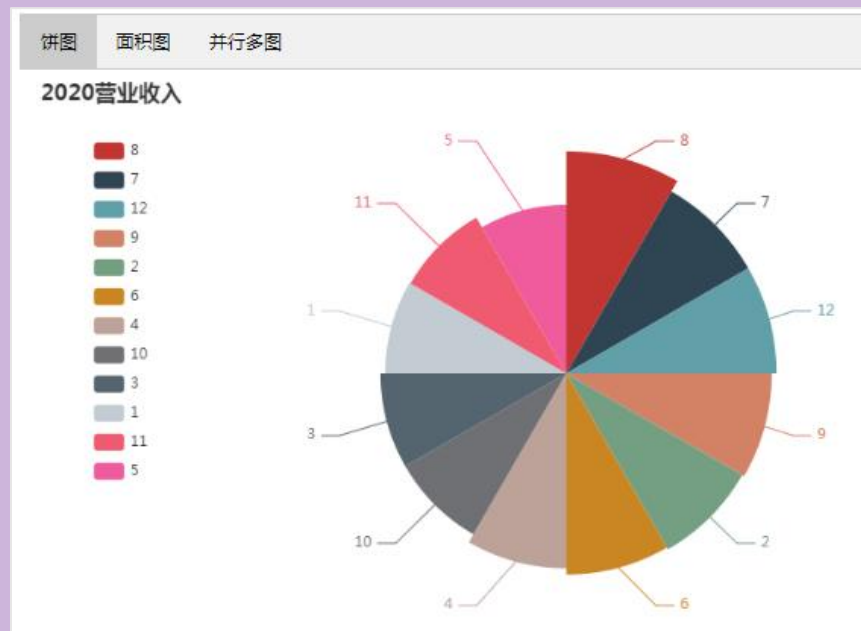
Tab：选项卡多图，仅需将图形逐一添加即可。

例4-2-12：将例4-2-8、例4-2-9、例4-2-10绘制的图形，使用Tab()函数组合在一起。

```
# 引入Tab
from pyecharts.charts import Tab

# 选项卡多图
tab = Tab()
tab.add(pie, '饼图')
tab.add(line1, '面积图')
tab.add(grid1, '并行多图')

# 展示图形
tab.render_notebook()
```



时间线轮播多图-Timeline

Timeline：时间线轮播多图，绘制各期图形后按时间线添加即可。

例4-2-13：绘制2019年营业收入饼图，并将例4-2-8绘制的2020年营业收入饼图按时间线组合。

```
# 引入Timeline
from pyecharts.charts import Timeline
```

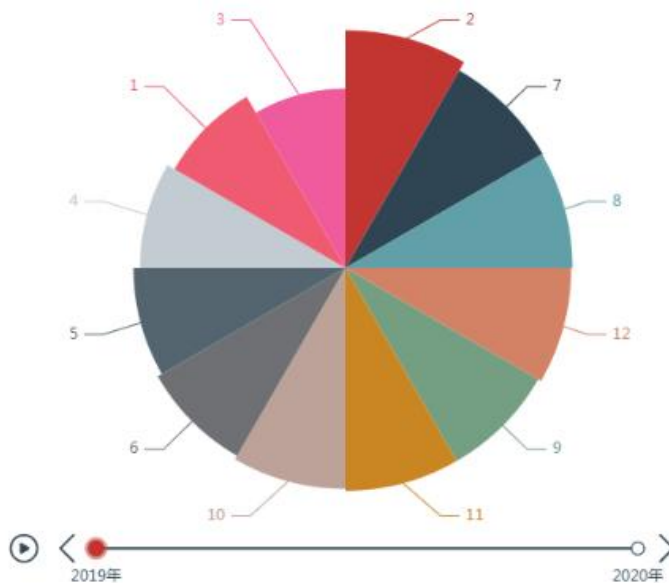
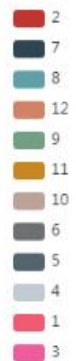
```
# 数据准备
```

```
df_2019 = df3.loc[df3['年']=='2019'].sort_values('营业收入',ascending=True)
df_2020 = df3.loc[df3['年']=='2020'].sort_values('营业收入',ascending=True)
```

```
# 绘制2019年饼图
```

```
x_2019 = df_2019['月'].tolist()
y_2019 = df_2019['营业收入'].tolist()
data_2019 = [z for z in zip(x_2019, y_2019)]
pie_2019 = Pie()
pie_2019.add('营业收入', data_2019)
pie_2019.set_global_opts(title_opts=opts.TitleOpts(title='2019年营业收入'))
```

2020营业收入



```
legend_opts = opts.LegendOpts(pos_left='5%',
```

```
vertical'))
```

```
tolist()
```

```
收入'].tolist()
```

```
p(x_2020,y_2020)]
```

```
data_2020,rosetype='area')
```

```
2019年')
```

```
2020年')
```

```
k()
```

财务案例实践

中淮公司旗下A子公司2020年度负债及权益项目金额如表所示。

项目	金额
流动负债	30000
非流动负债	30000
所有者权益	40000

通过Pyecharts绘制饼图展示A子公司的负债及所有者权益结构，饼图的内半径为90，外半径为150；饼图的中心（圆心）坐标参数为：center=["50%" , "50%"]。

```
from pyecharts.charts import Pie

attr = ["流动负债", "非流动负债", "所有者权益"]
value = [30000, 30000, 40000]

# 系列数据项，格式为 [(key1, value1), (key2, value2)]
data = [z for z in zip(attr,value)]

pie = Pie()
pie.add(
    "结构类型",data,
    # 饼图的中心（圆心）坐标,数组的第一项是横坐标,第二项是纵坐标
    center=["35%", "50%"],
    # 默认设置成百分比,设置成百分比时第一项是相对于容器宽度,第二项是相对于容器高度
    radius = [90,150]
)
pie.set_global_opts(
    title_opts=opts.TitleOpts(title="A子公司的负债及所有者权益结构"),
    legend_opts=opts.LegendOpts(pos_top=20)
)
pie.render_notebook()
```

财务案例实践

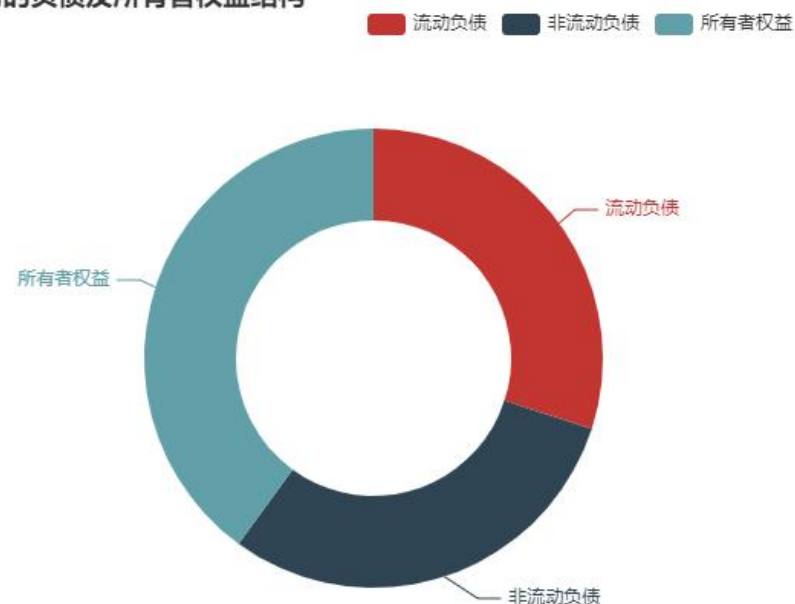
单独执行代码时，会报错 “NameError: name 'opts' is not defined”

需要在第二行重新引入options包，代码如下：

```
from pyecharts.charts import Pie
from pyecharts import options as opts
```



A子公司的负债及所有者权益结构



感谢观看